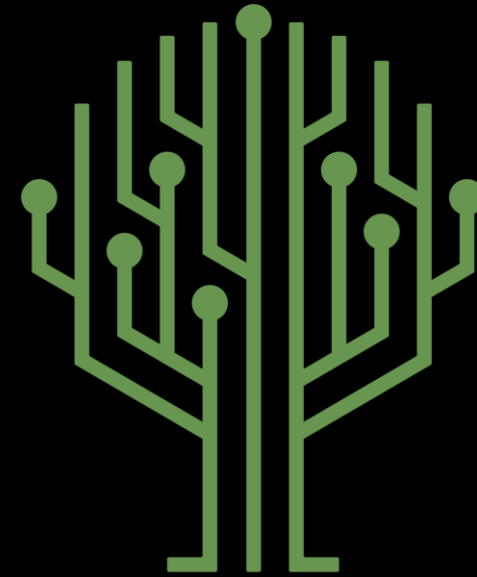


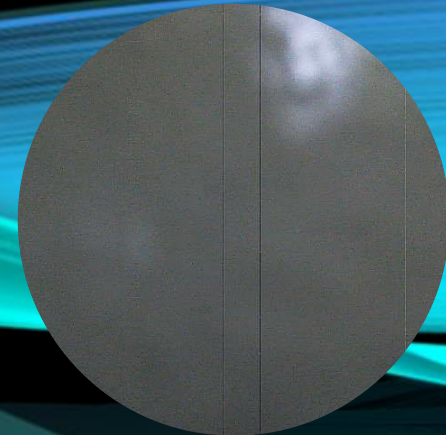
Green Pace

Security Policy Presentation
Developer: *Matthew Biletznikoff*

CS 405 Secure Coding
Project Two

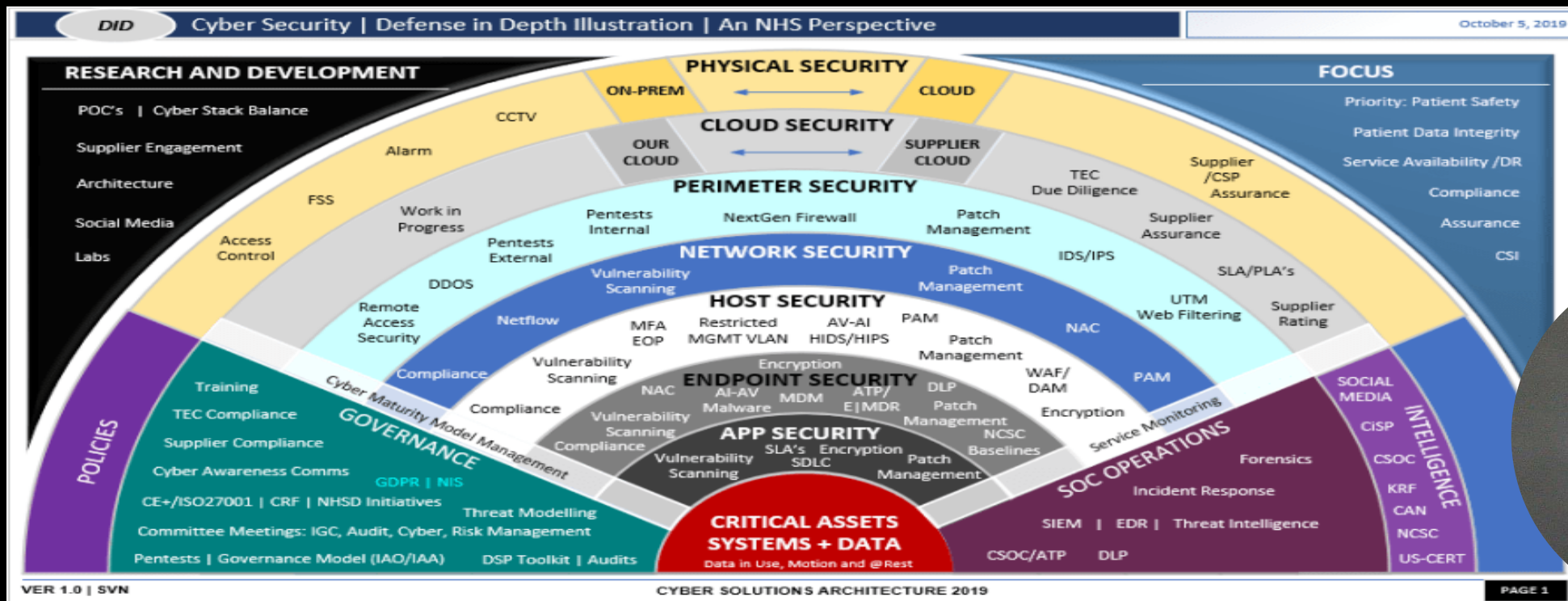


Green Pace



OVERVIEW: DEFENSE IN DEPTH

- Standardized security policy
- Defense-in-Depth security strategy
- DevSecOps automation and compliance



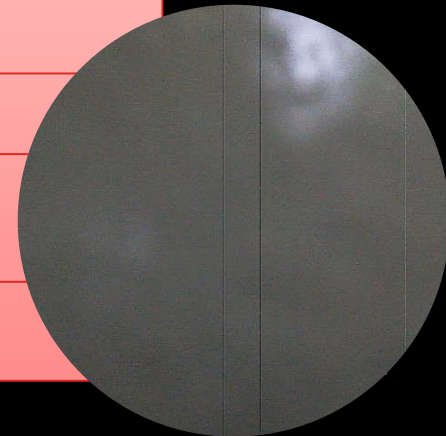
THREATS MATRIX

- **Threat Matrix Summary**
- Level 1 vulnerabilities require immediate remediation.
- Level 2 vulnerabilities should be addressed during normal development cycles.
- Level 3 vulnerabilities are monitored and remediated as resources permit.
- SQL Injection, Memory Protection, and Pointer Safety represent the greatest organizational risk.

Likely STD-004-CPP — SQL Injection STD-005-CPP — Memory Protection STD-008-CPP — Pointer Safety STD-002-CPP — Data Value STD-003-CPP — String Correctness STD-009-CPP — Resource Management	Priority Highest Priority Standards STD-004-CPP — SQL Injection STD-005-CPP — Memory Protection STD-008-CPP — Pointer Safety Highest organizational risk requiring immediate remediation.
Low Priority STD-001-CPP — Data Type STD-006-CPP — Assertions STD-007-CPP — Exceptions	Unlikely STD-010-CPP — File I/O

10 PRINCIPLES

Principle	Related Standards
1. Validate Input Data	STD-002, STD-003, STD-004, STD-010
2. Heed Compiler Warnings	STD-001, STD-005, STD-008
3. Architect and Design for Security Policies	STD-004, STD-007, STD-010
4. Keep It Simple	STD-006, STD-009
5. Default Deny	STD-004, STD-010
6. Principle of Least Privilege	STD-004, STD-010
7. Sanitize Data Sent to Other Systems	STD-003, STD-004, STD-010
8. Practice Defense in Depth	STD-001 through STD-010
9. Use Effective Quality Assurance Techniques	STD-001 through STD-010
10. Adopt a Secure Coding Standard	STD-001 through STD-010



CODING STANDARDS

Priority	Coding Standard	Vulnerability Area
1	STD-004-CPP	SQL Injection
2	STD-005-CPP	Memory Protection
3	STD-008-CPP	Pointer Safety
4	STD-002-CPP	Data Value
5	STD-003-CPP	String Correctness
6	STD-009-CPP	Resource Management
7	STD-001-CPP	Data Type
8	STD-006-CPP	Assertions
9	STD-007-CPP	Exceptions
10	STD-010-CPP	File I/O

- **Prioritization Method:**

Standards were ranked by severity, likelihood, remediation cost, priority, and overall threat level from the Green Pace risk assessment.

ENCRYPTION POLICIES

Encryption Policy	How It Is Applied	Example Controls
Encryption at Rest	Protects stored data in databases, files, backups, and cloud storage	AES-256, database encryption, encrypted backups
Encryption in Flight	Protects data moving between users, systems, APIs, and services	TLS 1.3, HTTPS, VPN tunnels, secure API communication
Encryption in Use	Protects data while actively processed by applications	Access controls, secure memory handling, key management, secure execution environments

Policy Goal:

Protect sensitive data throughout its full lifecycle: storage, transmission, and processing.

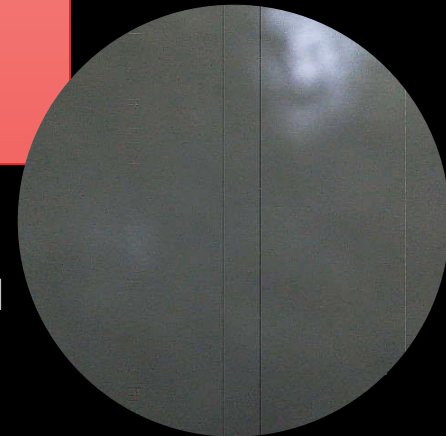


TRIPLE-A POLICIES

Policy	Purpose	Examples
Authentication	Verifies the identity of users and systems	User logins, passwords, MFA, service accounts
Authorization	Controls what authenticated users can access	Role-based access control, user permissions, file access restrictions
Accounting	Records and monitors system activity	Successful and failed logins, database modifications, new user creation, privilege changes, file access logs

Policy Goal:

- Ensure that only authorized users gain access to resources, operate within approved permissions, and generate auditable records of all security-relevant activities.



Unit Testing: Numeric Overflow Testing

```
Starting Numeric Underflow / Overflow Tests!

*****
*** Running Overflow Tests ***
*****
Overflow Test of Type = char
    Adding Numbers Without Overflow (0, 25, 5) = overflow: false, result: 125
    Adding Numbers With Overflow (0, 25, 6) = overflow: true, calculation prevented
Overflow Test of Type = wchar_t
    Adding Numbers Without Overflow (0, 13107, 5) = overflow: false, result: 65535
    Adding Numbers With Overflow (0, 13107, 6) = overflow: true, calculation prevented
Overflow Test of Type = short
    Adding Numbers Without Overflow (0, 6553, 5) = overflow: false, result: 32765
    Adding Numbers With Overflow (0, 6553, 6) = overflow: true, calculation prevented
Overflow Test of Type = int
    Adding Numbers Without Overflow (0, 429496729, 5) = overflow: false, result: 2147483645
    Adding Numbers With Overflow (0, 429496729, 6) = overflow: true, calculation prevented
Overflow Test of Type = long
    Adding Numbers Without Overflow (0, 429496729, 5) = overflow: false, result: 2147483645
    Adding Numbers With Overflow (0, 429496729, 6) = overflow: true, calculation prevented
Overflow Test of Type = __int64
    Adding Numbers Without Overflow (0, 1844674407370955161, 5) = overflow: false, result: 9223372036854775805
    Adding Numbers With Overflow (0, 1844674407370955161, 6) = overflow: true, calculation prevented
Overflow Test of Type = unsigned char
    Adding Numbers Without Overflow (0, 51, 5) = overflow: false, result: 255
    Adding Numbers With Overflow (0, 51, 6) = overflow: true, calculation prevented
Overflow Test of Type = unsigned short
    Adding Numbers Without Overflow (0, 13107, 5) = overflow: false, result: 65535
    Adding Numbers With Overflow (0, 13107, 6) = overflow: true, calculation prevented
Overflow Test of Type = unsigned int
    Adding Numbers Without Overflow (0, 858993459, 5) = overflow: false, result: 4294967295
    Adding Numbers With Overflow (0, 858993459, 6) = overflow: true, calculation prevented
Overflow Test of Type = unsigned long
    Adding Numbers Without Overflow (0, 858993459, 5) = overflow: false, result: 4294967295
    Adding Numbers With Overflow (0, 858993459, 6) = overflow: true, calculation prevented
Overflow Test of Type = unsigned __int64
    Adding Numbers Without Overflow (0, 3689348814741910323, 5) = overflow: false, result: 18446744073709551615
    Adding Numbers With Overflow (0, 3689348814741910323, 6) = overflow: true, calculation prevented
Overflow Test of Type = float
    Adding Numbers Without Overflow (0, 6.80565e+37, 5) = overflow: false, result: 3.40282e+38
    Adding Numbers With Overflow (0, 6.80565e+37, 6) = overflow: true, calculation prevented
Overflow Test of Type = double
    Adding Numbers Without Overflow (0, 3.59539e+307, 5) = overflow: false, result: 1.79769e+308
    Adding Numbers With Overflow (0, 3.59539e+307, 6) = overflow: true, calculation prevented
Overflow Test of Type = long double
    Adding Numbers Without Overflow (0, 3.59539e+307, 5) = overflow: false, result: 1.79769e+308
    Adding Numbers With Overflow (0, 3.59539e+307, 6) = overflow: true, calculation prevented
```

- Tests integer overflow conditions across multiple data types
- Uses automated boundary testing to identify arithmetic vulnerabilities
- Validates boundary checking before arithmetic operations
- Prevents unsafe calculations from executing
- Supports STD-001-CPP (Use Appropriate and Bounded Numeric Data Types)
- Supports Principle 1 (Validate Input Data)

Take It Further:

- Integrate overflow testing into CI/CD pipelines
- Execute automatically during the Verify stage
- Add fuzz testing for unexpected numeric inputs



Unit Testing: Numeric Underflow Testing

```
*****
*** Running Underflow Tests ***
*****
Underflow Test of Type = char
    Subtracting Numbers Without Underflow (-3, 25, 5) = underflow: false, result: -128
    Subtracting Numbers With Underflow (-3, 25, 6) = underflow: true, calculation prevented
Underflow Test of Type = wchar_t
    Subtracting Numbers Without Underflow (65535, 13107, 5) = underflow: false, result: 0
    Subtracting Numbers With Underflow (65535, 13107, 6) = underflow: true, calculation prevented
Underflow Test of Type = short
    Subtracting Numbers Without Underflow (-3, 6553, 5) = underflow: false, result: -32768
    Subtracting Numbers With Underflow (-3, 6553, 6) = underflow: true, calculation prevented
Underflow Test of Type = int
    Subtracting Numbers Without Underflow (-3, 429496729, 5) = underflow: false, result: -2147483648
    Subtracting Numbers With Underflow (-3, 429496729, 6) = underflow: true, calculation prevented
Underflow Test of Type = long
    Subtracting Numbers Without Underflow (-3, 429496729, 5) = underflow: false, result: -2147483648
    Subtracting Numbers With Underflow (-3, 429496729, 6) = underflow: true, calculation prevented
Underflow Test of Type = __int64
    Subtracting Numbers Without Underflow (-3, 1844674407370955161, 5) = underflow: false, result: -9223372036854775808
    Subtracting Numbers With Underflow (-3, 1844674407370955161, 6) = underflow: true, calculation prevented
Underflow Test of Type = unsigned char
    Subtracting Numbers Without Underflow (255, 51, 5) = underflow: false, result: 0
    Subtracting Numbers With Underflow (255, 51, 6) = underflow: true, calculation prevented
Underflow Test of Type = unsigned short
    Subtracting Numbers Without Underflow (65535, 13107, 5) = underflow: false, result: 0
    Subtracting Numbers With Underflow (65535, 13107, 6) = underflow: true, calculation prevented
Underflow Test of Type = unsigned int
    Subtracting Numbers Without Underflow (4294967295, 858993459, 5) = underflow: false, result: 0
    Subtracting Numbers With Underflow (4294967295, 858993459, 6) = underflow: true, calculation prevented
Underflow Test of Type = unsigned long
    Subtracting Numbers Without Underflow (4294967295, 858993459, 5) = underflow: false, result: 0
    Subtracting Numbers With Underflow (4294967295, 858993459, 6) = underflow: true, calculation prevented
Underflow Test of Type = unsigned __int64
    Subtracting Numbers Without Underflow (18446744073709551615, 3689348814741910323, 5) = underflow: false, result: 0
    Subtracting Numbers With Underflow (18446744073709551615, 3689348814741910323, 6) = underflow: true, calculation prevented
Underflow Test of Type = float
    Subtracting Numbers Without Underflow (0, 6.80565e+37, 5) = underflow: false, result: -3.40282e+38
    Subtracting Numbers With Underflow (0, 6.80565e+37, 6) = underflow: true, calculation prevented
Underflow Test of Type = double
    Subtracting Numbers Without Underflow (0, 3.59539e+307, 5) = underflow: false, result: -1.79769e+308
    Subtracting Numbers With Underflow (0, 3.59539e+307, 6) = underflow: true, calculation prevented
Underflow Test of Type = long double
    Subtracting Numbers Without Underflow (0, 3.59539e+307, 5) = underflow: false, result: -1.79769e+308
    Subtracting Numbers With Underflow (0, 3.59539e+307, 6) = underflow: true, calculation prevented

All Numeric Underflow / Overflow Tests Complete!

C:\Users\mbile\source\repos\NumericOverflows_Biletnikoff\x64\Debug\NumericOverflows.exe (process 59472) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Take It Further:

- Expand testing to floating-point edge cases
- Add automated regression testing
- Monitor arithmetic exceptions during runtime

- Tests arithmetic operations for underflow conditions
- Uses automated boundary testing to identify arithmetic vulnerabilities
- Validates lower boundary limits
- Prevents invalid calculations and unexpe
- Supports STD-001-CPP (Use Appropriate Numeric Data Types)
- Supports Principle 9 (Quality Assurance)

Unit Test: Numeric Boundary Validation

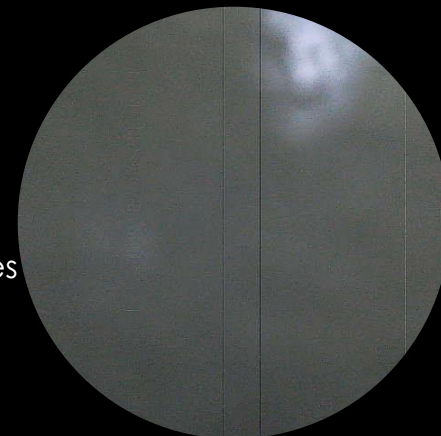
```
Subtracting Numbers Without Overflow (65535, 13107, 5) = underflow: false, result: 0
Subtracting Numbers With Overflow (65535, 13107, 6) = underflow: true, calculation prevented
Underflow Test of Type = unsigned int
Subtracting Numbers Without Overflow (4294967295, 858993459, 5) = underflow: false, result: 0
Subtracting Numbers With Overflow (4294967295, 858993459, 6) = underflow: true, calculation prevented
Underflow Test of Type = unsigned long
Subtracting Numbers Without Overflow (4294967295, 858993459, 5) = underflow: false, result: 0
Subtracting Numbers With Overflow (4294967295, 858993459, 6) = underflow: true, calculation prevented
Underflow Test of Type = unsigned __int64
Subtracting Numbers Without Overflow (18446744073709551615, 3689348814741910323, 5) = underflow: false, result: 0
Subtracting Numbers With Overflow (18446744073709551615, 3689348814741910323, 6) = underflow: true, calculation prevented
Underflow Test of Type = float
Subtracting Numbers Without Overflow (0, 6.80565e+37, 5) = underflow: false, result: -3.40282e+38
Subtracting Numbers With Overflow (0, 6.80565e+37, 6) = underflow: true, calculation prevented
Underflow Test of Type = double
Subtracting Numbers Without Overflow (0, 3.59539e+307, 5) = underflow: false, result: -1.79769e+308
Subtracting Numbers With Overflow (0, 3.59539e+307, 6) = underflow: true, calculation prevented
Underflow Test of Type = long double
Subtracting Numbers Without Overflow (0, 3.59539e+307, 5) = underflow: false, result: -1.79769e+308
Subtracting Numbers With Overflow (0, 3.59539e+307, 6) = underflow: true, calculation prevented

All Numeric Underflow / Overflow Tests Complete!

C:\Users\mbile\source\repos\NumericOverflows_Biletnikoff\x64\Debug\NumericOverflows.exe (process 42236) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Take It Further:

- Demonstrates comprehensive boundary testing
- Uses automated boundary testing to identify arithmetic vulnerabilities
- Verifies both upper and lower numeric limits
- Detects arithmetic vulnerabilities before deployment
- Supports defense-in-depth strategy
- Reduces risk of logic and arithmetic defects
- Supports STD-001-CPP (Bounded Numeric Data Types)
- Supports Principle 8 (Practice Defense in Depth)



Unit Testing: SQL Injection Detection

```
Microsoft Visual Studio
SQL Injection Example
Connected to the database.
USERS table created.

SQL: SELECT * from USERS ==> 4 records found.
User: Fred [UID=1 PWD=Flinstone]
User: Barney [UID=2 PWD=Rubble]
User: Wilma [UID=3 PWD=Flinstone]
User: Betty [UID=4 PWD=Rubble]

SQL: SELECT ID, NAME, PASSWORD FROM USERS WHERE NAME='Fred' ==> 1 records found.
User: Fred [UID=1 PWD=Flinstone]
Potential SQL injection detected. Query was blocked: SELECT ID, NAME, PASSWORD FROM USERS WHERE NAME='Fred' or 2=2;
Potential SQL injection detected. Query was blocked: SELECT ID, NAME, PASSWORD FROM USERS WHERE NAME='Fred' or 2=2;
Potential SQL injection detected. Query was blocked: SELECT ID, NAME, PASSWORD FROM USERS WHERE NAME='Fred' or 2=2;
Potential SQL injection detected. Query was blocked: SELECT ID, NAME, PASSWORD FROM USERS WHERE NAME='Fred' or 'hack'='hack';
Potential SQL injection detected. Query was blocked: SELECT ID, NAME, PASSWORD FROM USERS WHERE NAME='Fred' or 'hi'='hi';

C:\Users\mbile\source\repos\SQLInjection\x64\Debug\SQLInjection.exe (process 42616) exited with code 0 (0x0).
Press any key to close this window . . .
```

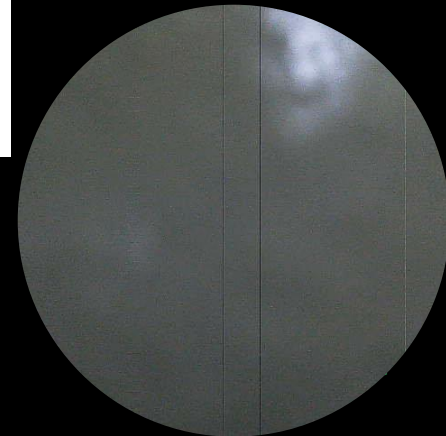
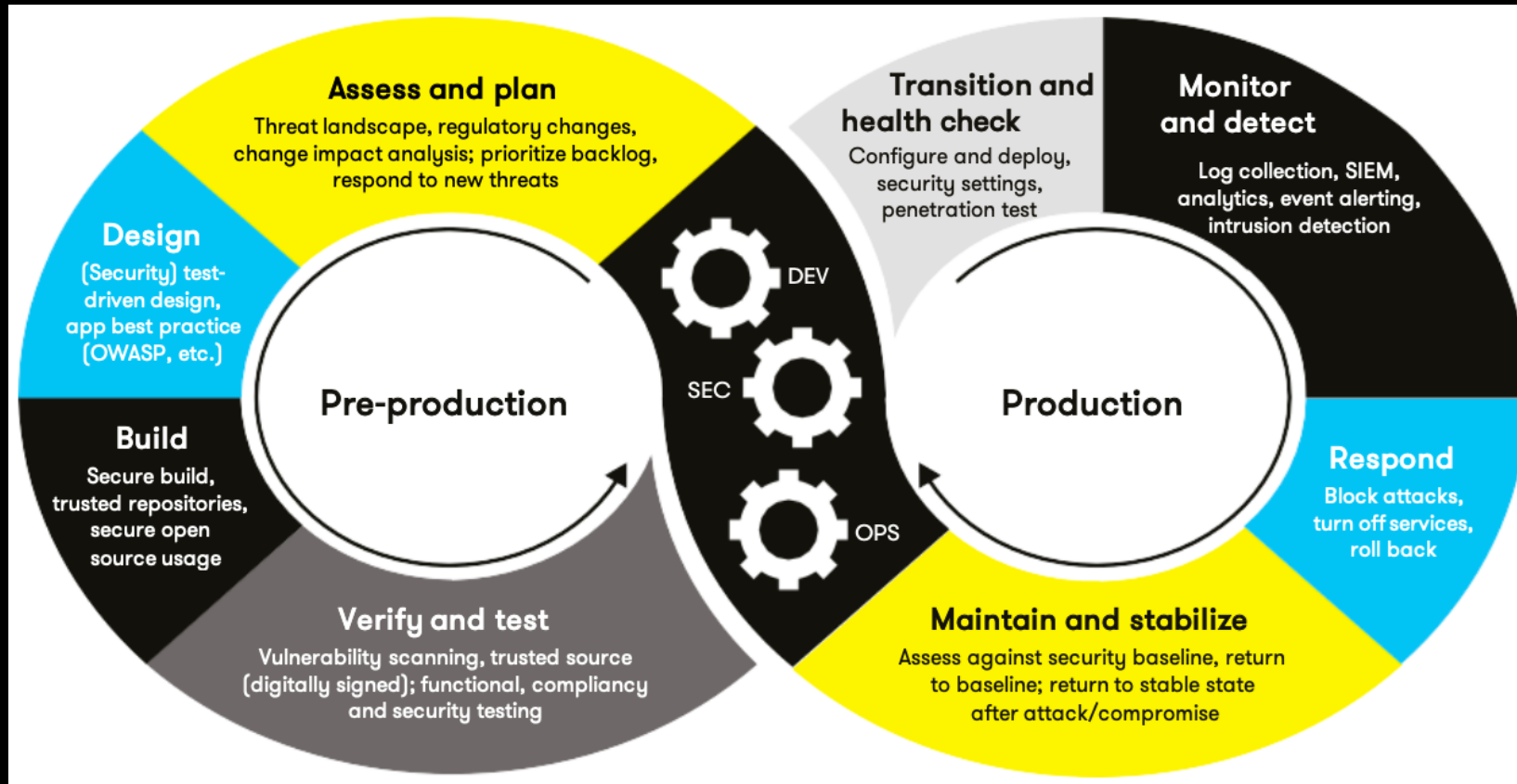
Take It Further:

- Integrate SQL injection testing into CI/CD pipelines
- Expand test cases using OWASP SQL injection payloads
- Automate database security scanning during Verify and Test
- Monitor blocked injection attacks through logs

- Tests application defenses against SQL injection attacks
- Uses malicious input patterns to validate input handling
- Detects and blocks unauthorized database queries
- Verifies secure query execution through input validation and parameterized queries
- Supports STD-004-CPP (SQL Injection Prevention)
- Supports Principle 1 (Validate Input Data)
- Demonstrates mitigation of a Level 1 security threat



AUTOMATION SUMMARY



DevSecOps Security Tools

Security Automation by Phase

Assess & Plan

- Threat modeling
- Security requirements

Design

- Secure architecture review
- OWASP / SEI CERT guidance

Build

- Compiler warnings
- Dependency scanning

Verify & Test

- Unit testing
- SQL injection testing
- Overflow/underflow testing
- Static analysis
- Cppcheck, Clang-Tidy, SonarQube
- Standards compliance validation

Transition and Health Check

- Security validation
- Configuration review

Monitor and Detect

- Log collection
- Security alerts
- Audit logging

Respond

- Incident response
- Patch management
- Vulnerability remediation

Maintain and Stabilize

- Policy updates
- Continuous improvement

Security automation is integrated throughout the DevSecOps lifecycle to detect vulnerabilities early, enforce Green PACE standards, and support a defense-in-depth security strategy.



RISKS AND BENEFITS

Vulnerability	Solution	Benefit of Acting Now	Risk of Delaying
SQL Injection	Input validation and parameterized queries	Prevents unauthorized database access and data theft	Increased risk of data breach and system compromise
Memory Protection	Boundary checking and safe memory management	Reduces crashes, corruption, and exploitation	Memory corruption vulnerabilities may remain exploitable
Pointer Safety	Null validation and smart pointers	Improves system stability and reliability	Application crashes and undefined behavior may occur
Numeric Overflow/Underflow	Boundary validation and automated testing	Prevents arithmetic errors and logic flaws	Incorrect calculations may lead to security vulnerabilities
Resource Management	RAII and smart pointers	Prevents resource leaks and improves performance	Memory leaks may reduce availability and reliability

Green Pace prioritizes mitigation of Level 1 vulnerabilities first because they present the highest risk to confidentiality, integrity, and availability. Early remediation reduces long-term costs and strengthens the organization's defense-in-depth strategy.

RECOMMENDATIONS

Current Policy Gaps

Security Monitoring

- Limited automated threat detection
- Security event correlation can be improved

Secure Development Training

- Developers require ongoing secure coding education
- Regular awareness training should be implemented

Third-Party Risk Management

- Open-source dependencies require continuous monitoring
- Vendor security reviews should be standardized

Penetration Testing

- External testing should occur on a scheduled basis
- Red-team exercises should be incorporated

Gap Analysis Summary:

Green Pace has established a strong security foundation, but future improvements should focus on automating monitoring, training, and adoption of industry security standards to proactively address emerging threats.

Future Security Improvements

Expand DevSecOps Automation

- Integrate security testing into every build
- Automate policy compliance validation

Adopt Industry Standards

- OWASP Secure Coding Practices
- SEI CERT C++ Coding Standard
- NIST Cybersecurity Framework

Continuous Vulnerability Management

- Routine scanning and remediation
- Automated security reporting and dashboards

Zero Trust Principles

- Verify every user and device
- Enforce least privilege access controls



CONCLUSIONS

Key Takeaways

- Security must be integrated throughout the software development lifecycle.
- Defense-in-depth provides multiple layers of protection against evolving threats.
- Secure coding standards reduce vulnerabilities before software reaches production.
- Automated testing and security automation improve consistency, efficiency, and compliance.
- Encryption, Triple-A controls, and continuous monitoring strengthen system security.

Final Recommendations

- Continue enforcing all Green Pace coding standards.
- Expand automated security testing across all development stages.
- Adopt industry frameworks such as OWASP, SEI CERT, and NIST.
- Conduct regular security reviews and penetration testing.
- Maintain a culture of continuous security improvement.

Closing Statement

Green Pace's security policy establishes a strong foundation for protecting confidentiality, integrity, and availability while supporting future growth through secure development practices, continuous monitoring, and proactive risk management.

Security is not a single control, it is a continuous process of prevention, detection, response, and improvement.



REFERENCES

Course Materials

Southern New Hampshire University. (2025). *CS 405 Secure Coding course materials.*

Industry Standards

OWASP Foundation. (2024). *OWASP secure coding practices.* Retrieved from <https://owasp.org>

Software Engineering Institute. (2024). *SEI CERT C++ coding standard.* Retrieved from <https://wiki.sei.cmu.edu>

National Institute of Standards and Technology. (2024). *Cybersecurity framework (CSF) 2.0.* Retrieved from <https://www.nist.gov>

Development and Testing Resources

[Microsoft Learn C++ Documentation](#)

[SQLite Documentation](#)

[OWASP SQL Injection Prevention Cheat Sheet](#)

Security Best Practices

Open Web Application Security Project. (2024). *OWASP Top 10 web application security risks.* Retrieved from <https://owasp.org>

